

SmartPortfolioManager

README / Project Overview

Blazor Web App + Entity Framework Core + SQLite + ASP.NET Core Identity

Project Objective

SmartPortfolioManager is a Blazor web application for managing and analyzing an investment portfolio. The project was developed as a mini-project for the .NET C# programming module.

The main idea of the application is to help an investor keep track of assets, buy/sell transactions, investment budget, and portfolio performance in one place.

- manage financial assets
- record buy and sell transactions
- keep an investment budget
- view transaction history
- analyze portfolio performance through a dashboard

The dashboard gives useful indicators such as current portfolio value, total invested amount, overall gain or loss, performance by asset, allocation by asset type, budget usage, and portfolio diversification health.

Main Features

Feature	Description
Asset Management	The application stores the assets that exist in the portfolio. Each asset contains a name, symbol, type, current price, creation date, and notes. Assets can be stocks or cryptocurrencies.
Transaction Tracking	The application records buy and sell transactions. Each transaction is linked to an asset and contains the transaction type, quantity, price, date, and notes. This keeps a clear history of portfolio movements.
Investment Budget	The budget represents the investment plan of the portfolio. It stores the initial budget amount, monthly contribution, and notes. The dashboard uses the budget to compare planned investment with the real invested amount.
Dashboard	The dashboard displays total assets, current portfolio value, total invested amount, gain/loss, budget usage, performance by asset chart, allocation by asset type chart, asset performance details, and portfolio health message.
Search and Filtering	Assets can be filtered by name, symbol, or type. Transactions can be filtered by asset name, transaction type, or selected asset. Filtering is done through service methods using LINQ and IQueryable.
Authentication and Authorization	The project uses ASP.NET Core Identity. Visitors can only access public pages. Users can consult the portfolio. Admin can manage assets, transactions, and budget.

Budget Usage Formula

The budget is used in the dashboard to compare the planned investment capacity with the real invested amount.

$$\text{Remaining Budget} = \text{Initial Budget} - \text{Total Invested}$$

Access Levels

Role	Allowed Access	Purpose
Visitor	Home, Login, Register	Can understand the app and create or access an account.
User	Assets, Transactions, Dashboard	Can consult portfolio data, search/filter, and analyze performance.
Admin	Everything User can access + add/edit/delete assets and transactions + budget management	Controls the financial data and manages the portfolio records.

Technologies Used

- C#
- Blazor Web App
- Entity Framework Core
- SQLite
- ASP.NET Core Identity
- LINQ
- Radzen Blazor charts
- Bootstrap

Project Structure

```
SmartPortfolioManager
|
|-- Components
|   |-- Layout
|   |-- Pages
|   |-- UI
|
|-- Data
|   |-- AppDbContext.cs
|
|-- Migrations
|
|-- Models
|   |-- Enums
|   |-- Asset.cs
|   |-- Transaction.cs
```

```

| |-- Budget.cs
| |-- PriceHistory.cs
| |-- AssetPerformanceStat.cs
| |-- AssetTypeAllocationStat.cs
|
|-- Services
| |-- IPortfolioService.cs
| |-- PortfolioService.cs
|
|-- wwwroot
|
|-- Program.cs
|-- appsettings.json
|-- SmartPortfolioManager.csproj

```

Folder Explanation

Folder	Role
Components	Contains the Blazor UI of the application.
Components/Pages	Contains the main screens such as Home, Login, Register, Assets, Transactions, Dashboard, Budget, EditAsset, and EditTransaction.
Components/UI	Contains reusable components such as KpiCard, AssetTable, and TransactionTable. These help avoid duplicated UI code.
Components/Layout	Contains the general layout and navigation menu. NavMenu controls sidebar links and hides Admin-only links from normal users.
Models	Contains the business classes used by EF Core to create database tables. The models also contain validation attributes such as Required, StringLength, and Range.
Data	Contains AppDbContext.cs, which connects EF Core and Identity to the SQLite database.
Services	Contains the business logic. Pages call IPortfolioService instead of accessing the database directly.
Migrations	Contains EF Core migration files used to create and update the database schema.
wwwroot	Contains static files such as CSS, Bootstrap files, images, and client-side assets.

Database

The project uses SQLite with Entity Framework Core. The database is generated from C# models using Code First migrations.

Table	Purpose
Assets	Stores each stock or crypto asset.
Transactions	Stores buy and sell movements linked to an asset.
Budgets	Stores the portfolio investment plan.
PriceHistories	Prepared table for storing historical prices over time.

Main Relationships

One Asset can have many Transactions. Each Transaction belongs to one Asset through AssetId. This relationship is important because dashboard calculations are based on buy and sell transactions.

Asset 1 ---- * Transactions

One Asset can also have many PriceHistory records. This prepares the project for future price evolution tracking.

Asset 1 ---- * PriceHistories

Budget is stored as one portfolio configuration. It is not linked to each transaction directly because it represents the investment plan, not a transaction history.

Important Calculations

Current Quantity = Bought Quantity - Sold Quantity
Current Value = Current Quantity x Current Asset Price
Total Invested = Sum of all Buy transactions
Gain/Loss = Current Portfolio Value - Total Invested
Remaining Budget = Initial Budget - Total Invested

How to Run the Project

```
git clone https://github.com/your-username/SmartPortfolioManager.git
cd SmartPortfolioManager
dotnet restore
dotnet build
dotnet watch
```

After running dotnet watch, open the local URL shown in the terminal.

Default Admin Account

The application seeds an Admin account for testing and demonstration.

Email: admin@portfolio.com
Password: Admin123!

What We Learned in This Project

- Blazor pages and routing
- Razor syntax and UI events
- component-based architecture
- reusable components with parameters and EventCallback
- Entity Framework Core and Code First migrations
- LINQ queries and IQueryable filtering
- asynchronous database access
- EditForm and validation with Data Annotations

- services and dependency injection
- dashboard indicators and charts
- authentication and role-based authorization

Future Improvements

- make each user manage only their own portfolio data using a UserId column
- allow Admin to manage portfolios by user
- add historical price charts using PriceHistory
- connect real market prices from an API
- improve budget analytics
- add export to PDF or Excel
- add notifications when the budget is exceeded

Work Division

This project was completed by Raed and Amira.

Raed

- worked mainly on the models
- worked on the data layer
- helped set up Entity Framework Core
- worked on part of the service logic

Amira

- worked mainly on the Blazor pages
- worked on the user interface
- worked on the dashboard design
- worked on authentication and authorization integration
- worked on search and filtering
- worked on budget page improvements
- handled final testing and GitHub preparation

Collaboration

Some parts were completed together, especially the general architecture, debugging, dashboard logic, and final project adjustments. Both members collaborated to make sure the application follows the project requirements and the notions studied in class.